```
class no_copies
{
public:
    no_copies(){}
    no_copies(no_copies const&) = delete;
    no_copies& operator=(no_copies const&) = delete;
};
```

This is much more descriptive than the original code and clearly expresses the intent. It also allows the compiler to give more descriptive error messages and moves the error from link time to compile time if you try to perform the copy within a member function of your class.

If, as well as deleting the copy constructor and copy-assignment operator, you also explicitly write a move constructor and move-assignment operator, your class becomes move-only, the same as `std::thread` and `std::unique_lock<>`. The following listing shows an example of such a move-only type.

Listing A.2. A simple move-only type

```
class move_only
{
    std::unique_ptr<my_class> data;
public:
    move_only(const move_only&) = delete;
    move_only(move_only&& other):
        data(std::move(other.data))
    {}
    move_only& operator=(const move_only&) = delete;
    move_only& operator=(move_only&& other)
    {
        data=std::move(other.data);
        return *this;
    }
};
move_only m1;
move_only m2(m1);
move_only m3(std::move(m1));
```

Error, copy constructor is declared deleted

OK, move constructor found

Move-only objects can be passed as function parameters and returned from functions, but if you wish to move from an lvalue, you always have to be explicit and use `std::move()` or a `static_cast<T&&>`.